Project Name: PathMate.ai

Team Members:

- **Dohyeon Lee** (dohyeon.lee@stonybrook.edu)
- **Heesang Kim** (heesang.kim@stonybrook.edu)

1. Introduction

1.1 Product Description

PathMate.ai is an AI-driven career management platform that integrates fragmented job preparation into one. Apart from traditional simple job posting arrangements, it deeply analyzes users' profiles (GPA, major, project, skill stack, etc.) to match optimal jobs, and provides data-driven support for the entire workflow from document preparation to interview practice.

1.2 Scope

Core Features:

- Provides a career dashboard based on user competency data
- Job analysis and personalized matching scores (Fit Score)
- Create and calibrate customized cover letter drafts using AI
- Hiring Simulator and Feedback Report for Practice AI

1.3 Target Users

Graduating Seniors
Students entering the job market who need immediate, realistic matching for their first professional roles.

Active Job Seekers
Individuals facing difficulties in the current employment market seeking a competitive edge.

Early-Career Planners
High school or early university students looking to build a strategic roadmap for their future career paths.

2. Requirements

Since the previous SRS phase and UI Mockup announcement, we have redefined the requirements around the following four key functions.

2.1 Functional Requirements

FR-1: The system should display a 'Readiness Score' on the main screen, which scores the user's current readiness.

FR-2: Based on the activity history of the user, the 'Weekly Tasks' to be completed this week and the full career roadmap should be visualized and provided.

FR-3: A 'Fit Score' should be calculated by comparing the user's profile with the job posting.

FR-4: Not only the score, but also the 'Match Logic' and organizational culture suitability and pay indicators of why this job was matched should be detailed.

FR-5: Custom interview questions should be generated based on selected job postings and user profiles.

FR-6: Multiple feedback reports such as Structure, Impact, and Clarity should be provided by analyzing the user's answers.

FR-7: Create a draft cover letter linking the key keywords in the job description with the user's experience.

FR-8: Users should be informed of the lack of job-related evidence in the prepared document through the 'Missing Evidence Flag' and encouraged to supplement it.

2.2 Non-functional Requirements

Performance: Key analysis results and AI responses must be completed within 15 seconds.

Security: Provides secure authentication with Google OAuth, and encrypts and manages personal competency data.

Usability: Minimize language barriers by providing Korean and English UI switching functions.

3. System Architecture

3.1 Overview

(1) Overview

Based on the **Client-Server architecture** based on **Next.js 16.1.6**, the system provides efficient data and authentication management through **Supabase (PostgreSQL 16)** and advanced AI services through **OpenAI (GPT-4o)** interworking. In particular, by tightly combining design and business logic using **Tailwind CSS 4.0** and Server Actions, it has a full-stack structure that ensures high scalability and development efficiency without the need for separate server infrastructure operations.

(2) Third-party Components & Frameworks

Key third-party technologies utilize **OpenAI SDK (v6.29.0)** to handle core AI matching and question generation logic, and implement intuitive UI icons with **Lucide React (v0.5777.0)**. In addition, clsx and tailwind-merge are combined to provide flexibility in styling and maximize the maintenance of code.

(3) Design Patterns

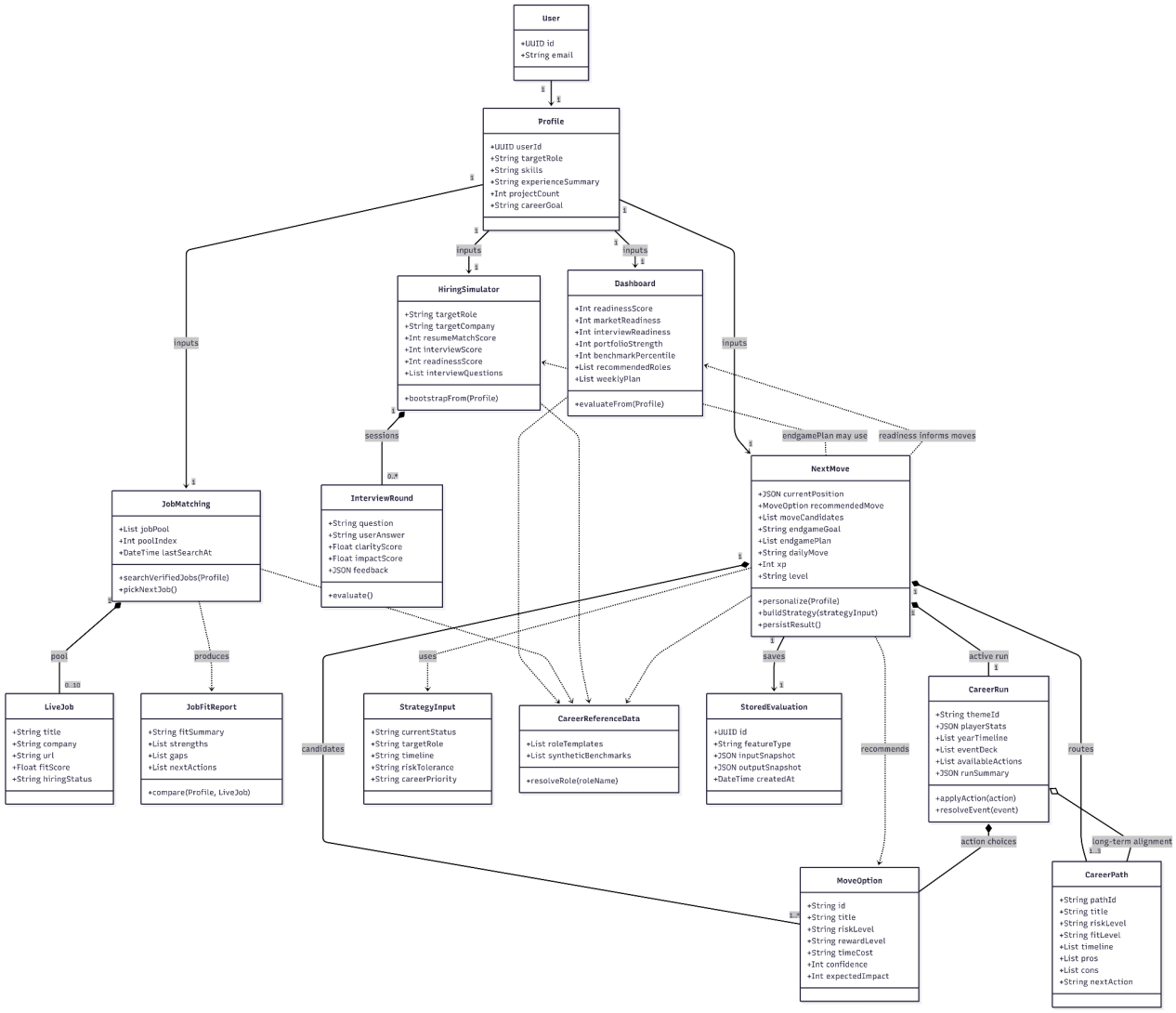
In terms of software design, MVC patterns are applied to clearly separate and manage data (Supabase), UI (React), and logic (Server Actions), and UI is designed in reusable units through a component-based architecture to increase development efficiency and code quality at the same time.

(4) Rationale

The integrated stack based on Next.js and TypeScript accelerates development, and utilizes Vercel and Supabase to create a cost-effective, high-performance infrastructure. By combining OpenAI APIs, we maximize the accuracy of matching and feedback, which is the core of the service.

3.2 Data Design

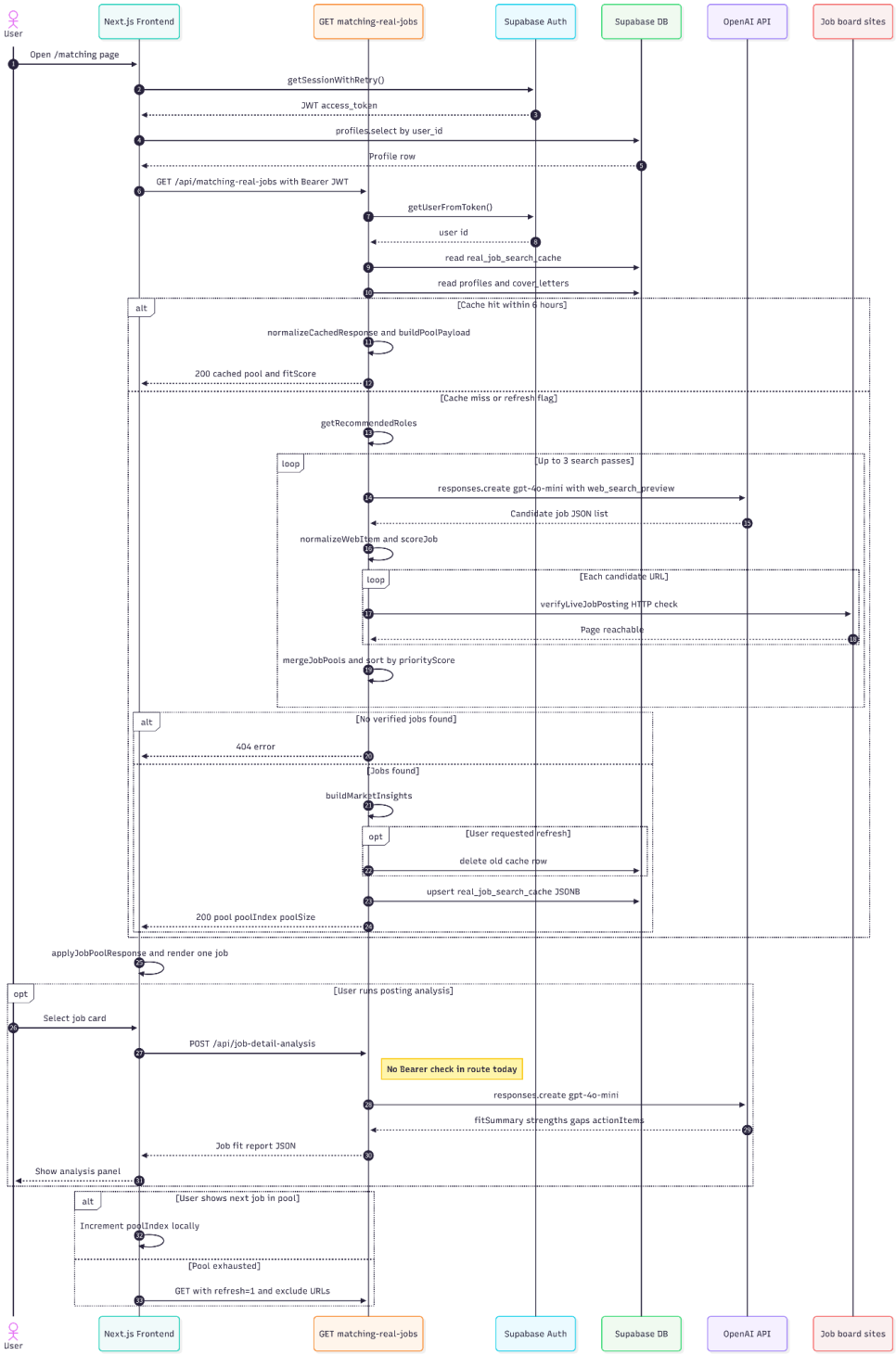
This UML class diagram outlines the core data structure behind PathMate.ai. These classes basically act as the bridge between the database (Supabase) and the Next.js frontend.



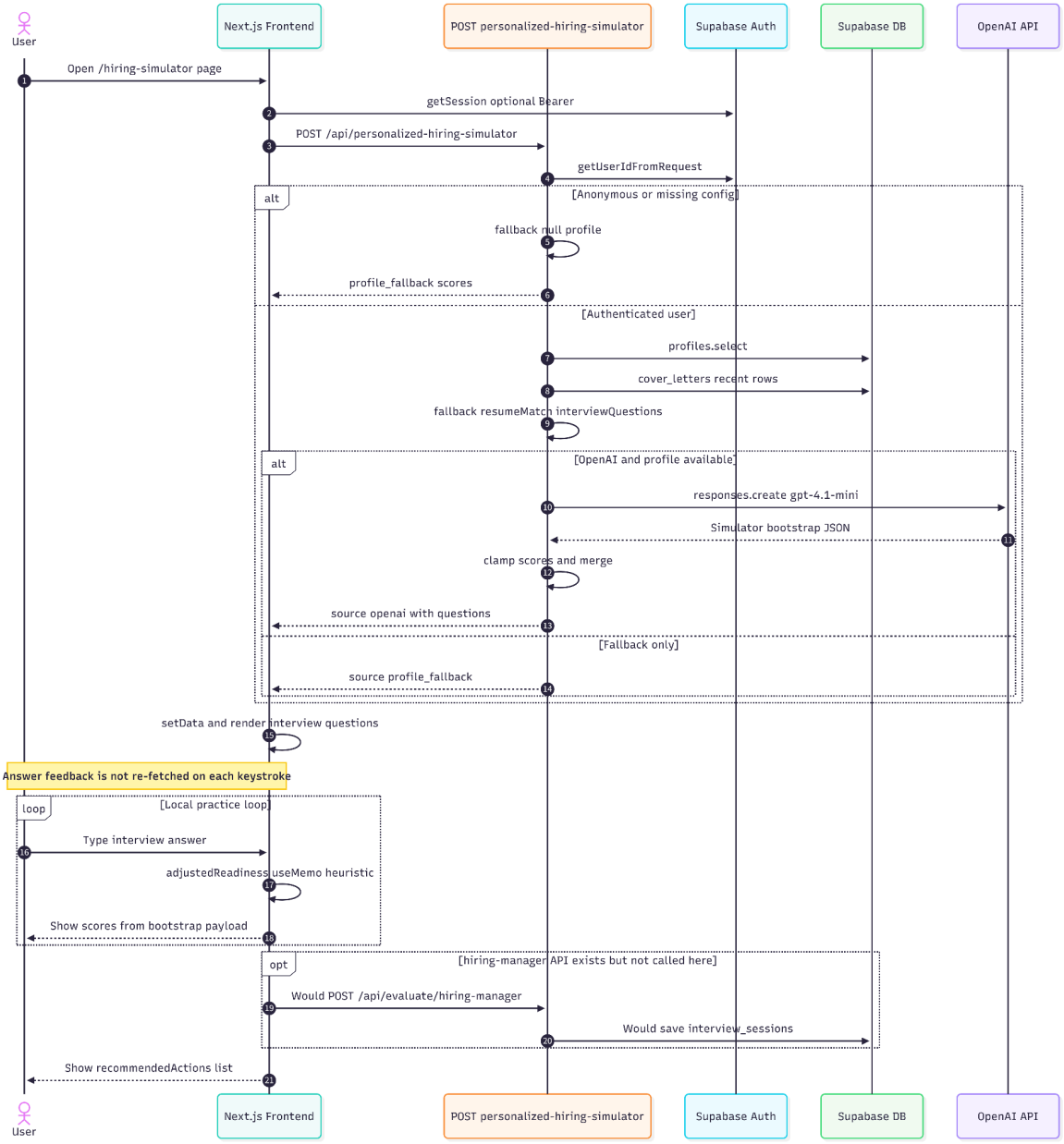
UML Class Diagram

3.3 UML Sequence Diagram

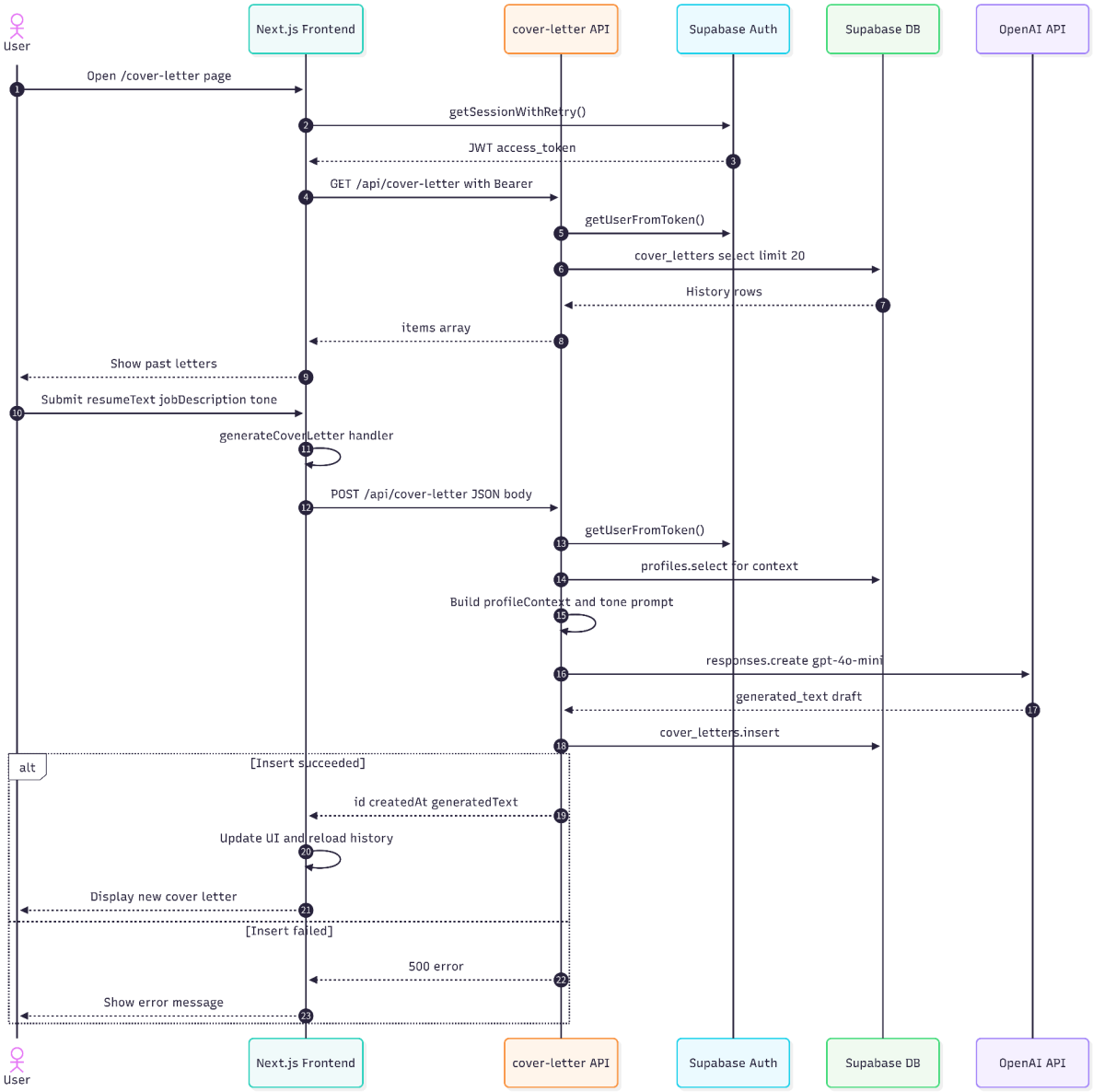
Case 1: Job Matching



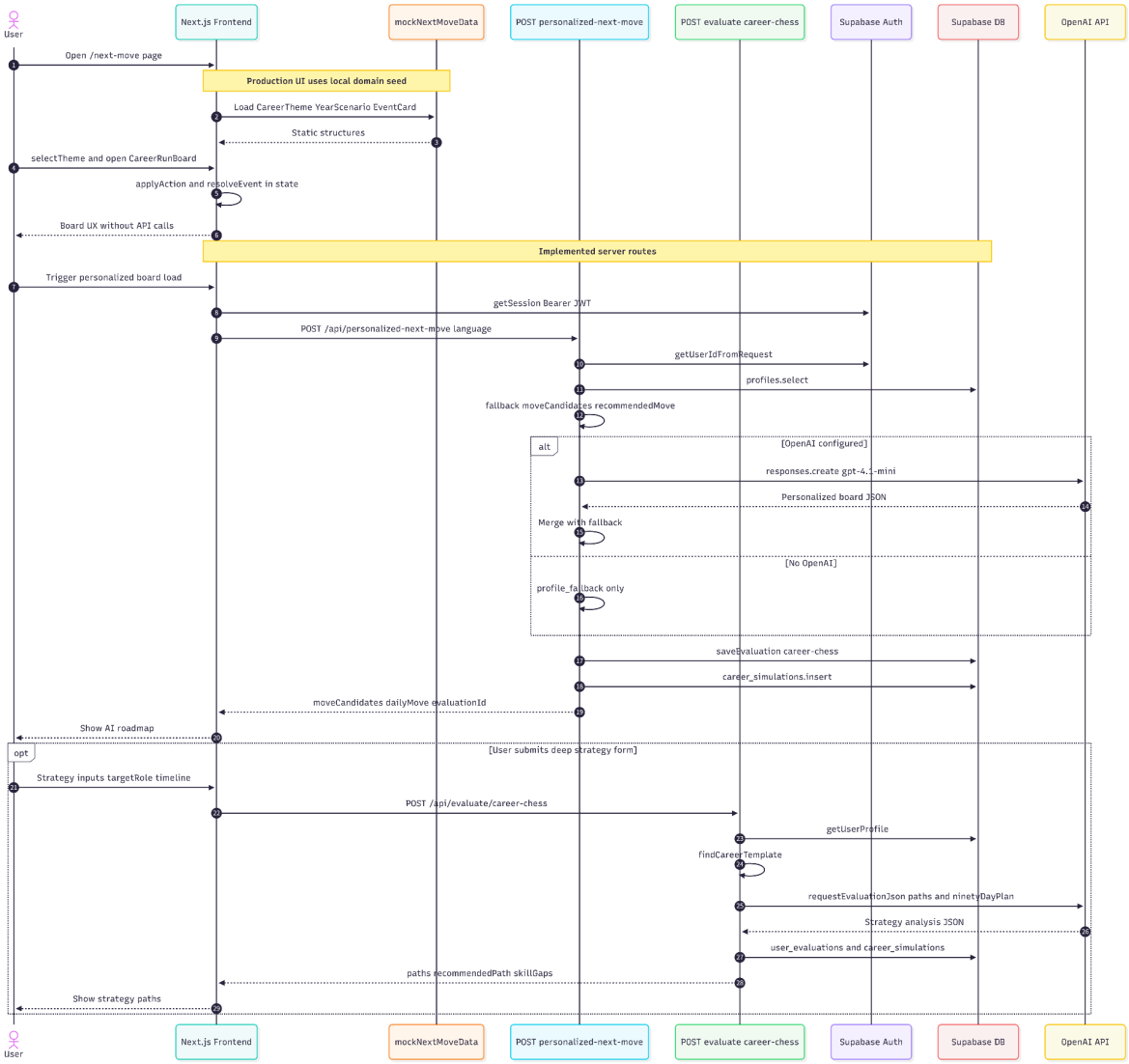
Case 2: Hiring Simulator



Case 3: Cover Letter



Case 4: Next Move



3.4 API Design

	Category	Method	Endpoint URL	Input (Request Body)	Output (Response Body)	Description
2	**Auth**	POST	/api/auth/signup	email, password	id, email, createdAt	Registers a new user and creates an entry in the authentication system.
3	**Auth**	POST	/api/auth/login	email, password	access_token, refresh_token	Authenticates credentials and returns a secure session token.
4	**Auth**	POST	/api/auth/logout	-	{ ok: true }	Invalidates the current session and signs the user out.
5	**Auth**	POST	/api/account/delete	-	{ ok: true }	Permanently deletes user credentials from Auth and purges all profile data.
6	**Career Dashboard**	GET	/api/dashboard-insight	?language=ko|en	name, topPercent, scores, weeklyReport	Retrieves user's career analytics and weekly performance report for the dashboard.
7	**Job Matching**	GET	/api/matching-real-jobs	?language, ?refresh, ?exclude, ?pooll	source, searchedAt, poolSize, items[], marketInsights	Fetches real-time job postings filtered by user profile using OpenAI web search analysis.
8	**Job Matching**	POST	/api/job-detail-analysis	jobTitle, company, summary, profileSt	fitSummary, strengths[], gaps[], actionItems[]	Provides a deep-dive comparison and gap analysis between a specific job posting and the user's profile.
9	**Cover Letter**	GET	/api/cover-letter	-	items[{id, generated_text, ...}]	Retrieves the generation history of all cover letters for the authenticated user.
10	**Cover Letter**	POST	/api/cover-letter	resumeText, jobDescription?, tone?, l	id, createdAt, generatedText	Generates a tailored AI cover letter based on user's resume and target job requirements.
11	**Setting**	GET	/api/job-alerts	-	items[{id, company, title, postedAt, url}]	Provides demo job alert cards based on the user's 'desired_companies' preference.
12	**Hiring Simulator**	POST	/api/personalized-hiring-simulator	language?	targetRole, readinessScore, interviewQuestions{}, recommendedActions[]	Initializes a personalized mock interview session and calculates initial readiness scores.
13	**Hiring Simulator**	POST	/api/evaluate/background-check	submittedLinks[] or submittedConten	overallScore, status, criticalIssues[], warnings[], strengths[], cleanupChecklist[]	Assesses professional reputation and risks based on user-submitted public profile links or text.
14	**Hiring Simulator**	POST	/api/simulator	mode=postings, profileSummary?, ta	items[](virtual postings)	Generates synthetic/virtual job postings for career simulation purposes (Internal).
15	**Hiring Simulator**	POST	/api/simulator	mode=question, targetRole, level	question, rubric[], followUps[]	Generates context-aware interview questions and evaluation rubrics (Internal).
16	**Hiring Simulator**	POST	/api/simulator	mode=evaluate, question, answer, tar	score, summary, hiringSignal	Evaluates a user's interview answer against the specific rubric and role (Internal).
17	**Hiring Simulator**	POST	/api/evaluate/hiring-manager	selectedPersona, role, question, user	overallScore, passProbability, strengths[], evaluationId?	Provides interview feedback from the perspective of a specific hiring manager persona (Internal).
18	**Career Dashboard**	POST	/api/evaluate/career-readiness	targetRole?, language?	overallScore, components{6 types}, strengths[], gaps[]	Generates a comprehensive snapshot of career readiness across 6 core dimensions (Internal).
19	**Next Move**	POST	/api/evaluate/career-chess	currentStatus, targetRole, timeline, ri	summary, recommendedPath, paths[], ninetyDayPlan[]	Generates strategic career paths and a detailed 90-day execution plan (Internal).
20	**Next Move**	POST	/api/personalized-next-move	language?	currentPosition{}, moveCandidates[], recommendedMove{}, xp, level	Processes gamified career roadmap and progression logic for the Next Move UI (Internal).
21	**System Data**	GET	/api/career-data/roles	?language=ko|en	roles[]	Retrieves a list of all supported job role templates available in the system (Internal).
22	**System Data**	GET	/api/career-data/roles/[role]	Path: [role], ?language	template, syntheticExamples[]	Fetches detailed template data and synthetic profile examples for a specific role (Internal).
23						
24						
25						

https://docs.google.com/spreadsheets/d/1jdC7KclbSgH-g5ZK_Q97vcSWbMoOt216kPUMrPtamA0/edit?gid=1110915406#gid=1110915406

3.5 Deployment

Our project combines **Vercel**'s automatic build and deployment pipeline with **Supabase**'s cloud database and authentication infrastructure to perform advanced AI analysis by linking **OpenAI API**s. In particular, Vercel and Supabase utilize the free plans of each service and solve the cost of using OpenAI through separate support to efficiently build and deploy the entire system infrastructure without incurring any cost.

3.6 Code Conventions

Complies with the Google JavaScript Style Guide for code readability and maintenance. The naming conventions apply camelCase (variable method), PascalCase (class), UPPER_SNAKE_CASE (constant) and are based on 2-space indentation, semi-colon usage, and lower quotation marks. It also automatically manages code style consistency among team members by introducing ESLint and Prettier.

4. Schedule

Milestone 1: Design Review (Completed: Early May)

Goal: Complete Software Design Document and determine system architecture.

Tasks:

- Software Design Document Finalization: Complete Document Structure and Text Organization (Heesang Kim) — *Completed on May 3, 2026*
- UML Diagram Precision: Class and Sequence Diagram Final Update (Dohyeon Lee) — *Completed on May 5, 2026*
- API Design Completed: Detailed API Specification with Google Sheet (Heesang Kim) — *Completed on May 6, 2026*

Milestone 2: Alpha Release (Completed: Mid-May)

Goal: Build basic infrastructure and complete User Interface framework.

Tasks:

- Data API endpoints: Profile data CRUD and dashboard statistics deployment API (Heesang Kim) — *Completed on May 11, 2026*
- Common UI components: Create a button, input form library for reuse across the globe (Dohyeon Lee) — *Completed on May 13, 2026*
- Core UI Layout: Career Cockpit Dashboard and Profile Management Page UI Development (Dohyeon Lee) — *Completed on May 15, 2026*

Milestone 3: Poster Presentation (Completed: Late May)

Goal: Visualize system architecture, complete the poster showcase, and gather initial peer evaluation.

Tasks:

- Poster presentation material production: Poster completion visualizing system architecture and core flows (Together) — *Submitted on May 17, 2026*
- AI Backend Logic: OpenAI API integration matching algorithm and interview question generation engine enhancement (Heesang Kim) — *Completed on May 22, 2026*
- Interactive AI workspace: Interview simulation interface and cover letter editing workspace check (Dohyeon Lee) — *Completed on May 24, 2026*
- Poster Presentation Showcase: Conducted the live poster presentation session and collected initial peer evaluation (Together) — *Presented on May 27, 2026*

Milestone 4: Beta Release & Progress Check-In (Completed: Early June)

Goal: Logic optimization, final multi-module system integration, and official website beta deployment.

Tasks:

- Logic Optimization: AI Prompt Engineering Advancement and Matching Logic Exception Handling (Heesang Kim) — *Completed on May 28, 2026*
- Integration & Vercel Deployment: Final system integration, deployment, and official Milestone 4 Beta Release via Vercel (Together) — *Completed on May 31, 2026*
- Milestone 4 Progress Update Documentation**:** Compile individual and group progress reports for professor check-in (Together) — *Completed on June 1, 2026*

Milestone 5: Final Release (In-Progress / Remainder Schedule)

Goal: Reflect user feedback, eliminate outstanding bugs, optimize performance, and deliver final deliverables.

Tasks:

- [In-Progress / Adjusted QA Workflow] Comprehensive Bug Tracking & Final Check: Execute mutual cross-validation based on Beta feedback. Dohyeon is tracking and fixing frontend layout bugs via GitHub Issues while Heesang handles continuous production re-deployments and data integrity checks (Together) — *Scheduled: May 29 – June 4, 2026*
- [In-Progress] Optimizing and Securing Performance: Improved API response speed, enhanced database indexing, and data security reinforcement (Heesang Kim) — *Scheduled: May 30 – June 4, 2026*
- [In-Progress] UI/UX Polishing: Add smooth micro-animations, improve asynchronous loading statuses, and conduct a final mobile responsive layout check (Dohyeon Lee) — *Scheduled: May 30 – June 4, 2026*
- Final Presentation: Supplement final functionality based on collected feedback and deliver the final live project presentation to the instructor (Together) — *Target Date: June 5, 2026*
- Final Report: Documentation of project performance and future development directions (Together) — *Target Date: June 10, 2026*

5. Contribution

Dohyeon Lee :

- Drafted the introduction and requirements sections for the Software Design Document.
- Served as the Lead Frontend Developer and UI Designer for the project.
- Developed the core user interface using Next.js and Tailwind CSS, including the Homepage, Profile management, and Dashboard.
- Designed and added new website design elements for the Hiring Simulator and AI Cover Letter Workspace to enhance user interaction.
- Implemented key functional requirements such as job filtering and real-time interview interfaces.
- Integrated high-fidelity dummy data to evaluate the system in a realistic service environment.
- Led the creation of high-fidelity mockups and UI flow diagrams to improve system visualization.

Heesang Kim :

- Drafted the system architecture sections for the Software Design Document.
- Served as the Lead Backend Architect and System Designer for the project.
- Designed the comprehensive database schema, entity relationships, and the detailed Data Dictionary.
- Defined the RESTful API specifications, including endpoint mapping and integration logic for OpenAI and Supabase.
- Designed UML Class and Sequence Diagrams to effectively model system behavior and data flow.
- Managed the GitHub repository, configured the Supabase infrastructure, and handled continuous deployment via Vercel.
- Finalized the technical design reports and updated the README with system setup and integration instructions.